

软件体系结构课程报告

IronSync-3D：智能健身看板与补剂管理系统的架构设计与实现

学生姓名：谭欣瑞

学号：14234802170

专业：软件工程

提交日期：2026 年 5 月 20 日

项目地址：<https://github.com/stephen14120123/IronSync-3D>

摘要

IronSync-3D 是一个面向健身爱好者的智能训练看板与补剂管家系统。本报告以该项目的架构设计为主线，首先记录了与 AI 助手就层级架构选型的探讨过程，随后运用"4+1 架构视图模型"对系统的逻辑结构、开发组织、运行时行为和物理部署进行多维度描述。前端采用原生 JavaScript + SVG 构建 2.5D 数据可视化看板，后端基于 Spring Boot 3.2 + MyBatis 提供 RESTful 服务。报告重点分析了**状态模式**在补剂库存预警中的设计应用，以及从 Three.js 到原生 SVG 的**架构降级权衡决策**，展示了在资源约束下以"合适的技术"替代"热门的技术"所获得的架构收益。此外，报告实录了 AI 辅助编码的对话过程，并附带完整的系统使用说明书。

关键词：软件体系结构；4+1 视图；状态模式；架构降级；前后端分离；AI 辅助开发

1. 前置准备与场景探讨 (Preparation)

1.1 与 AI 助手讨论层级架构适用性

以下记录了我在选题阶段与 AI 助手就架构风格选型进行的完整讨论过程。

我：你好，我最近在学《软件体系结构》这门课，讲义里讲到了分层架构（Layered Architecture）。我有点拿不准——这种架构到底适合什么样的项目？它有什么明显的优缺点吗？

AI：这是一个很好的切入点。分层架构的核心思想是"关注点分离"——每一层只负责一个特定的职责，上层依赖下层，下层对上层提供服务接口。它的适用场景和特点可以这样总结：

适合的场景：

- 以数据流转为核心的业务系统**——典型的 CRUD 应用，如电商后台、OA 系统、管理系统。这类系统的核心流程就是"接收请求 → 处理业务 → 读写数据库 → 返回响应"，分层架构天然匹配这个流程。
- 团队协作开发**——不同开发者可以专注于不同层（前端工程师写 Controller，后端工程师写 Service，DBA 优化 Mapper），职责边界清晰。
- 需要长期维护的企业级应用**——因为层与层之间解耦，替换某一层的实现（例如将 MyBatis 换成 JPA）理论上不影响其他层。

不适合的场景：

1. **高吞吐、低延迟的实时系统**——每经过一层就有一次方法调用和上下文切换的开销，对于游戏服务器、高频交易系统来说这个开销不可接受。
2. **极简的微服务或脚本**——如果业务逻辑只有几十行代码，硬套三层架构反而增加了不必要的复杂度。

主要优点：

- 抽象层次清晰，新成员上手快
- 层间通过接口通信，可替换性强
- 便于分层测试（可以分别 Mock 上层或下层）

主要缺点：

- 随着系统增长，层与层之间可能发生“泄漏”——比如业务逻辑渗透到 Controller 层或 SQL 散落在 Service 层
- 对于简单的查询操作，经过三层转发显得有些笨重

我：我最近在构思一个个人项目——想做一个轻量级的“智能健身看板与补剂管理系统”，主要功能是记录力量训练数据、管理补剂库存、展示一些可视化图表。你觉得这种系统适合用 Spring Boot 的三层架构（Controller/Service/Mapper）来做吗？

AI：非常适合。我们来分析一下这个系统的数据流特征：

你的系统核心操作无非是以下几种：

- | | | | | | |
|---|----------|------------------------------|------------|-------------|--------------|
| 1 | 用户录入训练记录 | → POST /api/training-records | → 校验参数 | → 计算统计数据 | → INSERT 数据库 |
| 2 | 用户查看看板 | → GET /api/dashboard/summary | → 聚合查询 | → 返回 JSON | → 前端渲染 |
| 3 | 用户管理补剂 | → PUT /api/supplements/{id} | → 重新计算剩余天数 | → UPDATE 状态 | → 返回结果 |

这个流程恰好是“请求 → 业务处理 → 数据持久化 → 响应”的标准三层模式。

为什么 Spring Boot 三层架构特别适合你的项目：

1. **职责天然对应：**你的训练 CRUD、补剂状态计算、饮食情绪录入等功能都是独立的业务单元，可以很自然地分配到 Service 层。Controller 只负责 HTTP 协议层面的参数校验和响应封装，Mapper 只负责 SQL。
2. **状态机逻辑需要独立的 Service 组件：**你说到补剂库存要自动预警——“充足/偏低/告急”的判定逻辑如果散落在 Controller 或 Mapper 里都会造成耦合。三层架构允许你在 Service 层内部再拆分出一个 StatusMachine 组件专门负责状态计算，这是分层思想的自然延伸。
3. **前端可能迭代替换：**你说想做可视化，现在可能是 ECharts，以后可能换 D3.js 甚至 WebGL。因为你们是前后端分离的，前端怎么换都不影响后端的三层结构。
4. **单人开发的边界管理：**即使是单人项目，三层架构也能帮你建立心理边界——你在写 Controller 的时候不需要想 SQL 怎么写，在写 Mapper 的时候不需要想 HTTP 状态码怎么处理，认知负载被有效分摊了。

需要注意的一点：因为你是单人开发，不要过度设计。三层架构本身已经提供了足够的结构，不需要再加 CQRS、事件溯源等重型模式。这一点恰好符合你“轻量级”的项目定位。

我：好，那我就用 Spring Boot + MyBatis 三层架构了。另外我想在仪表盘上做一个 2.5D 的人体部位可视化，目前的想法是用 Three.js，但我又担心它太重了，你怎么看？

AI：这部分我们可以在后续迭代中具体讨论，但你的直觉是对的——Three.js 对于数据看板来说通常是过度设计。我们先把这个议题放一放，先把三层架构的骨架搭起来，等实现到可视化模块时再具体看性能数据和替代方案。

我：同意。那项目的名字就定为 IronSync-3D，先做第一轮迭代把核心 CRUD 和三层架构跑通。

小结：经过上述讨论，我确定了 IronSync-3D 项目的技术选型——后端采用 Spring Boot 三层架构（Controller / Service / Mapper），前端采用原生 HTML/CSS/JS，前后端通过 RESTful API 通信。后续章节将详细描述这一架构的具体设计和实现过程。

2. 引言与需求分析

2.1 项目背景

在现代健身管理中，训练者通常需要记录力量训练数据、追踪补剂库存、管理饮食摄入与情绪状态，并将这些数据以可视化的方式进行综合分析。然而，市面上的健身管理应用存在两个极端：功能单一仅记录训练，或过于臃肿包含社交和课程等非核心功能。基于此，IronSync-3D 项目旨在构建一个轻量级的、聚焦数据分析的个人健身管理平台。

2.2 功能需求

系统需提供以下主要功能：

- 数据可视化看板：**以 2.5D 几何人体为载体，将训练数据以部位高亮方式直观展示
- 训练记录管理：**力量训练数据的 CRUD 操作，支持按动作筛选与分页查询
- 补剂库存管家：**补剂信息的增删改查，以及基于剩余天数的自动预警
- 饮食情绪关联：**宏量营养素摄入与情绪评分的关联趋势分析
- 身体指标追踪：**体重与体脂率的时序记录与趋势可视化
- MCP 本地检索：**基于关键词的本地 Markdown 笔记全文检索

2.3 非功能需求

类别	指标	目标值
性能	首屏加载时间	≤ 3 s
性能	API P95 响应时间	≤ 300 ms
性能	并发用户支持	≥ 500 VUs
可靠性	错误率	< 0.1%
安全性	鉴权机制	JWT Token 验证
安全性	密码存储	BCrypt 加盐哈希
安全性	SQL 注入防御	MyBatis 参数化查询
可维护性	前端依赖	零第三方运行时依赖
可部署性	部署方式	Docker 容器化

3. 系统架构风格设计

3.1 架构风格选型

IronSync-3D 采用**混合架构风格**，融合了以下三种基本风格：

- (1) **B/S 架构 (Browser/Server)**

系统采用浏览器/服务器模式。用户通过浏览器访问前端静态页面，所有业务逻辑与数据存储集中在服务端处理。该风格的优势在于零客户端安装、集中式维护与更新，契合现代 Web 应用的主流范式。

(2) 前后端分离架构

前端与后端通过 RESTful API 进行通信，二者在开发、部署、扩展三个维度完全解耦：

- **开发解耦**：前端仅关注 UI 渲染与交互，后端仅关注业务逻辑与数据持久化，二者可并行开发
- **部署解耦**：前端静态资源可由 Nginx 或 Spring Boot 内嵌容器托管，后端作为独立 Java 进程运行
- **扩展解耦**：前端无状态特性使其可通过 CDN 加速，后端可通过水平扩展实例应对流量增长

(3) 分层架构 (Layered Architecture)

后端严格遵循三层架构模式，自顶向下依次为：

- **Controller 层 (表现层)**：接收 HTTP 请求、参数校验、响应封装
- **Service 层 (业务逻辑层)**：业务规则编排、事务管理、状态计算
- **Mapper 层 (数据访问层)**：MyBatis 参数化查询、数据持久化

3.2 架构风格选择理由

上述三种风格的组合选择基于以下工程考量：

决策	权衡维度	理由
B/S 而非 C/S	部署成本 vs 交互体验	健身数据管理属低频使用场景，B/S 免安装的优势远大于 C/S 的交互优势
前后端分离 而非单体 MVC	开发效率 vs 架构复杂度	分离架构使得前端可视化层可独立迭代（如 Three.js 到 SVG 的切换不改后端一行代码）
三层而非 DDD 或 CQRS	架构复杂度 vs 业务复杂度	系统业务逻辑以 CRUD 为主，三层架构以最低的认知成本满足了设计需求，DDD 等重型风格在此场景下属于过度设计

3.3 系统架构总览

以下架构图展示了系统的整体结构与数据流向：

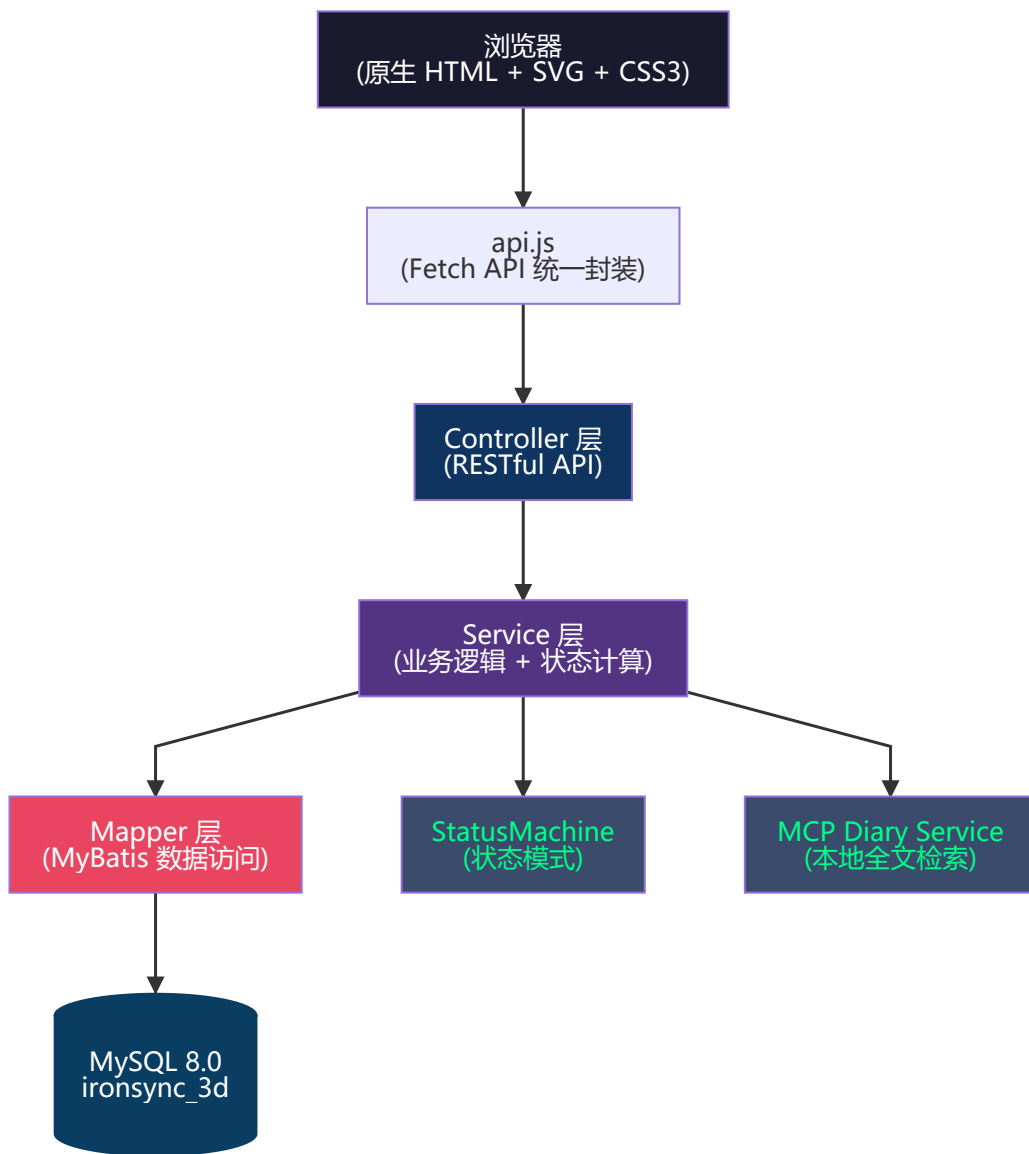


图 1：系统架构总览图

4. 4+1 架构视图

Philippe Kruchten 提出的"4+1 架构视图模型"从逻辑、开发、过程和部署四个相互正交的视角描述软件架构，并结合场景视图验证各视图的一致性。本节依此模型对 IronSync-3D 进行多维度架构描述。

4.1 逻辑视图 (Logical View)

逻辑视图描述系统的功能分解与模块划分，关注"做什么"。

4.1.1 功能模块划分

系统按业务领域划分为七个功能模块：

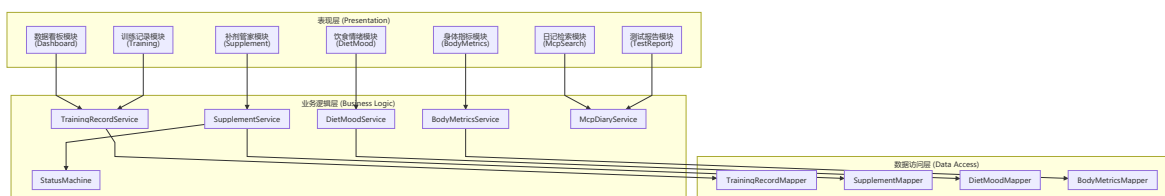


图 2：逻辑视图——功能模块划分与依赖关系

4.1.2 模块职责说明

模块	职责	关键接口
数据看板	聚合今日训练统计与热量消耗数据	<code>GET /api/dashboard/summary</code>
训练记录	力量训练数据的 CRUD 与趋势分析	<code>GET/POST/PUT/DELETE /api/training-records</code>
补剂管家	补剂库存管理与状态预警	<code>GET/POST/PUT/DELETE /api/supplements</code>
饮食情绪	宏量营养素与情绪评分的关联记录	<code>GET/POST /api/diet-mood</code>
身体指标	体重与体脂率的时序数据管理	<code>GET/POST /api/body-metrics</code>
日记检索	本地 Markdown 文件的全文检索	<code>GET /api/mcp/search</code>

4.2 开发视图 (Development View)

开发视图从代码组织角度描述系统的静态结构，关注"如何组织代码"。

4.2.1 后端包结构

后端采用 Maven 标准目录结构，包划分遵循"按层分包 + 按功能分包"的混合策略：

1	com.ironsync	
2	├─ IronSyncApplication.java	# 启动入口
3	├─ common/	# 通用基础设施
4	│ └─ result/Result.java	# 统一响应体泛型
5	│ └─ exception/	# 全局异常处理
6	│ └─ BusinessException.java	
7	│ └─ GlobalExceptionHandler.java	
8	│ └─ ErrorCode.java	
9	│ └─ interceptor/AuthInterceptor.java	# JWT 鉴权拦截器
10	│ └─ auth/TokenManager.java	# Token 管理
11	│ └─ constant/ActionEnum.java	# 动作枚举
12	├─ config/	# 配置类
13	│ └─ McpConfig.java	# MCP 目录配置
14	│ └─ MeshMappingConfig.java	# 部位映射配置
15	│ └─ webMvcConfig.java	# CORS / 静态资源
16	├─ entity/	# 数据实体 (5 张表)
17	├─ dto/	# 数据传输对象
18	│ └─ request/	# 请求 DTO (含 @valid 校验)
19	│ └─ response/	# 响应 VO
20	├─ mapper/	# MyBatis Mapper 接口 + XML
21	├─ service/	# 业务服务层
22	│ └─ impl/	# 服务实现类
23	│ └─ supplement/	
24	│ └─ StatusMachine.java	# 状态机计算
25	│ └─ StatusLevel.java	# 状态枚举

```

26 |   └─ mcp/McpDiaryService.java      # MCP 检索服务
27 |   └─ mesh/MeshMappingService.java  # 动作-部位映射
28 └─ controller/                     # REST 控制器
29     └─ AuthController.java          # 登录认证
30     └─ TrainingRecordController.java
31     └─ SupplementController.java
32     └─ DietMoodController.java
33     └─ BodyMetricsController.java
34     └─ DashboardController.java
35     └─ McpController.java
36     └─ TestReportController.java

```

4.2.2 前端资源结构

前端采用纯原生技术栈（HTML + CSS + JavaScript），无框架依赖：

```

1  static/
2  └─ index.html                    # 数据看板主页
3  └─ login.html                    # 登录页
4  └─ training.html                 # 训练记录页
5  └─ supplements.html              # 补剂管家页
6  └─ diet-mood.html                # 饮食情绪页
7  └─ body-metrics.html             # 身体指标页
8  └─ mcp-search.html              # 日记检索页
9  └─ test-report.html              # 测试报告页
10 └─ css/
11     └─ global.css                # 全局变量与重置
12     └─ dashboard.css             # 看板专用样式
13 └─ js/
14     └─ api.js                    # Fetch API 统一封装
15     └─ dashboard.js              # SVG 映射引擎逻辑

```

4.2.3 包依赖关系

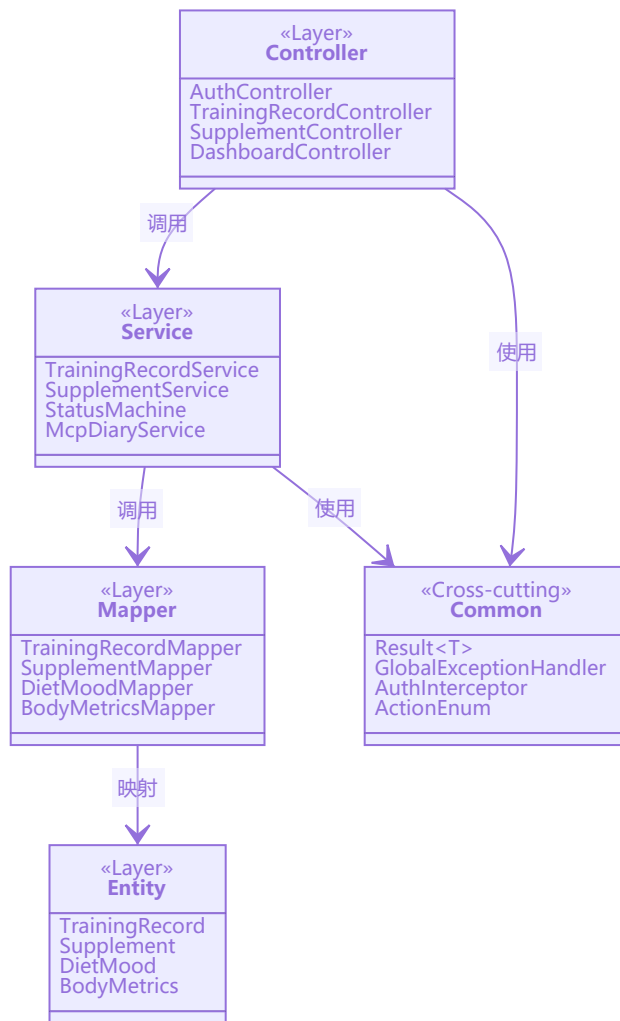


图 3：开发视图——包依赖关系

4.3 过程视图 (Process View)

过程视图描述系统的运行时行为，关注"如何运行"。本节以 JWT 鉴权流程和补剂预警流程为例说明关键运行时交互。

4.3.1 JWT 鉴权时序

系统采用无状态 JWT 鉴权方案，用户登录后服务端签发 Token，后续请求通过 `AuthInterceptor` 拦截校验。

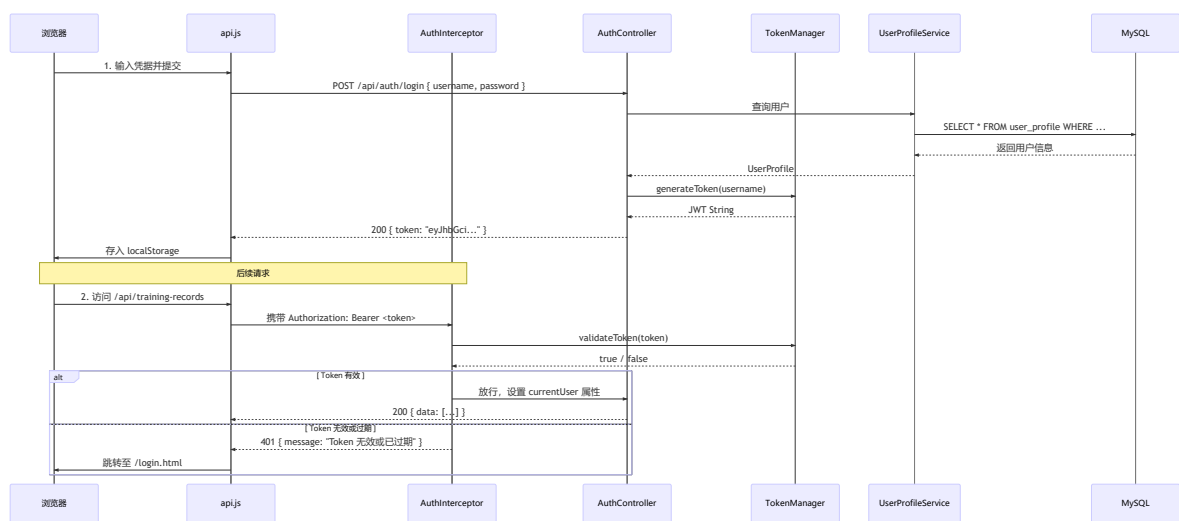


图 4: JWT 鉴权时序图

4.3.2 补剂库存预警过程

补剂库存预警在每次新增或编辑操作后触发，由 `StatusMachine` 组件执行状态判定。

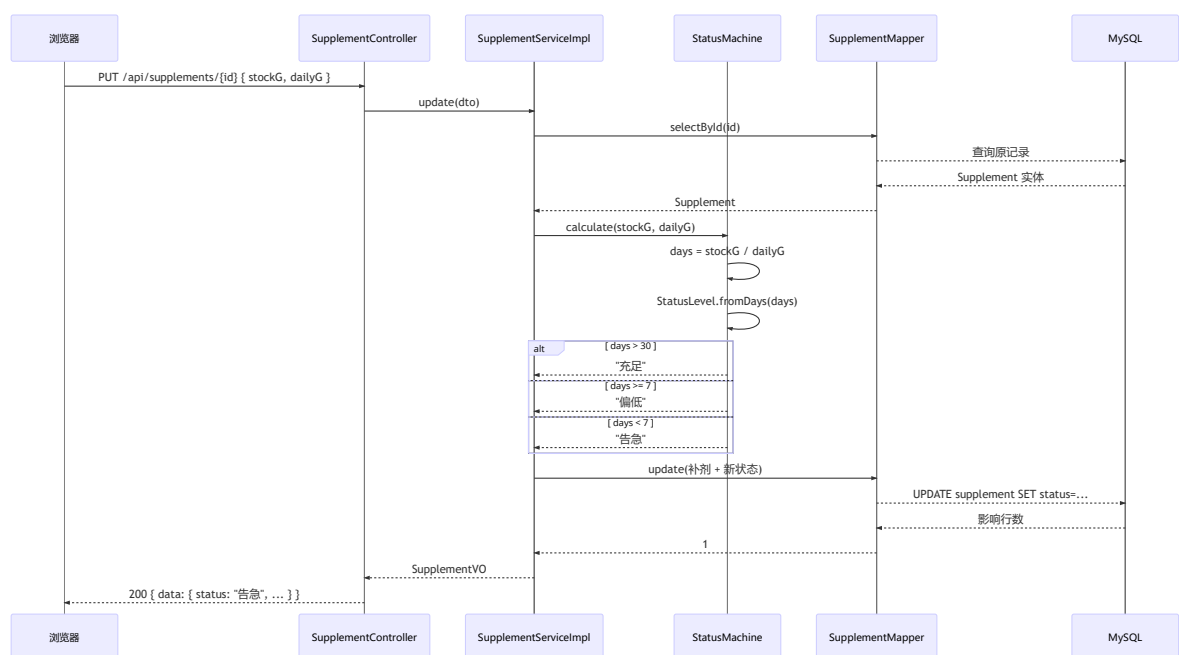


图 5：补剂库存预警过程时序图

4.4 物理视图 (Physical View)

物理视图描述系统的物理部署方案，关注"部署在哪里"。

4.4.1 Docker 容器化部署方案

系统采用 Docker Compose 进行多容器编排，共包含两个容器：

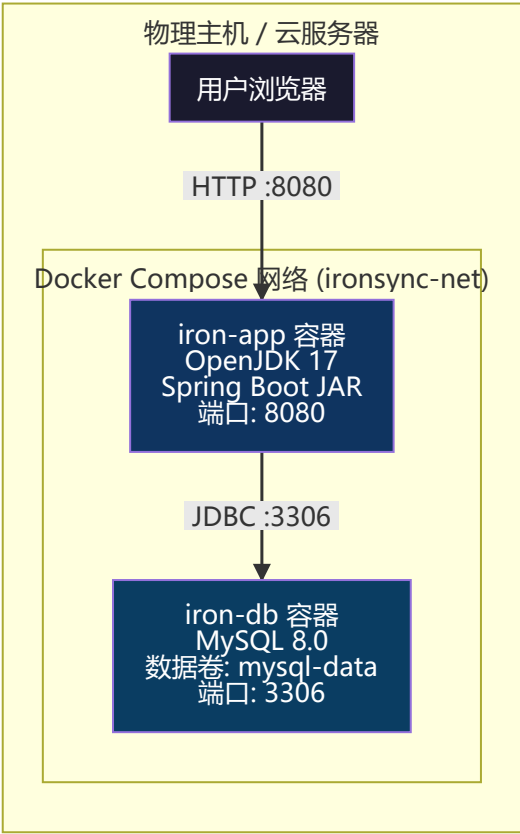


图 6：物理部署视图

服务配置要点：

- **iron-app**：基于 `openjdk:17-jdk-slim` 构建，通过 `Dockerfile` 将 Maven 构建产物 (JAR 包) 打包为镜像，容器内部暴露 8080 端口
- **iron-db**：基于 `mysql:8.0` 官方镜像，挂载 `deploy/init-db.sql` 实现初始化建表与种子数据导入，持久化数据存储于命名卷 `mysql-data`
- **网络**：两容器桥接于 `ironsync-net`，应用通过容器名 `iron-db` 而非 IP 地址访问数据库，实现服务发现
- **健康检查**：应用容器配置 `depends_on` 条件与 `healthcheck`，确保数据库就绪后再启动应用

4.4.2 部署流水线

每次部署按以下标准化流程执行：

- 1 [1] 构建阶段: `mvn clean package -DskipTests` → 产出 JAR
- 2 [2] 镜像阶段: `docker build` → 构建 `iron-app` 镜像
- 3 [3] 启动阶段: `docker-compose up -d` → 拉起 MySQL + 应用
- 4 [4] 验证阶段: `curl /api/health` → 预期 200
- 5 [5] 测试阶段 (可选): `npx playwright test` → 生成报告

5. 设计模式与架构权衡决策

5.1 状态模式在补剂预警中的应用

5.1.1 问题背景

补剂库存预警的核心逻辑为：根据 `当前库存量 / 每日消耗量` 计算剩余可用天数，依此将补剂标记为"充足" (>30 天)、"偏低" (7-30 天) 或"告急" (<7 天)。初版实现中，该逻辑散布于业务方法的 `if-else` 嵌套中：

```
1 // 反例：散落在业务方法中的阈值判断
2 if (days.compareTo(new BigDecimal("30")) > 0) {
3     status = "充足";
4 } else if (days.compareTo(new BigDecimal("7")) >= 0) {
5     status = "偏低";
6 } else {
7     status = "告急";
8 }
```

该实现存在三个问题：一是阈值常数（30、7）散落各处，修改需全局搜索替换；二是新增状态（如"即将耗尽"）需修改所有出现此逻辑的方法；三是缺少兜底保障，条件覆盖不全时将跳过所有分支。

5.1.2 状态模式设计

针对上述问题，引入状态模式（State Pattern）对预警逻辑进行重构。核心设计包含两个类：

StatusLevel 枚举 —— 将每个状态编码为一个带谓词条件的枚举常量：

```
1 public enum StatusLevel {
2     SUFFICIENT("充足", d -> d.compareTo(new BigDecimal("30")) > 0),
3     WARNING ("偏低", d -> d.compareTo(new BigDecimal("7")) >= 0),
4     CRITICAL("告急", d -> true); // 兜底条件：确保穷尽性
5
6     private final Predicate<BigDecimal> condition;
7
8     public static StatusLevel fromDays(BigDecimal days) {
9         for (StatusLevel level : values()) {
10             if (level.condition.test(days)) return level;
11         }
12         return CRITICAL;
13     }
14 }
```

StatusMachine 组件 —— 作为 Spring 组件，承担纯计算职责：

```
1 @Component
2 public class StatusMachine {
3     public String calculate(BigDecimal stockG, BigDecimal dailyConsumptionG)
4     {
5         BigDecimal days = stockG.divide(dailyConsumptionG, 2,
6             RoundingMode.HALF_UP);
7         return StatusLevel.fromDays(days).getDisplayName();
8     }
9 }
```

两个类的关系如下类图所示：

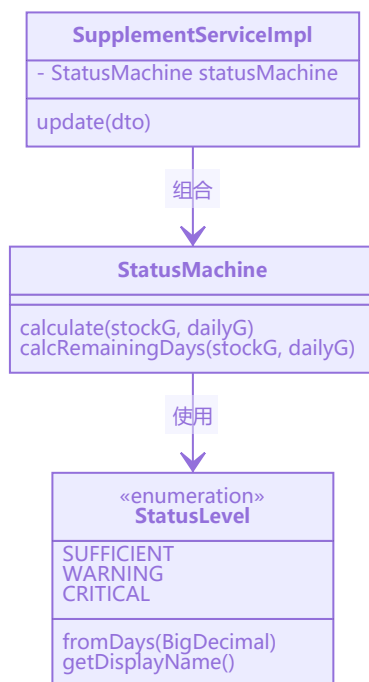


图 7：状态模式类图

5.1.3 模式收益分析

评价维度	if-else 实现	状态模式实现
开闭原则	新增状态需修改 N 处业务方法	新增枚举常量即可，业务方法零修改
阈值管理	魔法数字散落在各方法中	阈值集中定义在枚举构造函数中
穷尽性保障	人工保证，易遗漏	编译器通过枚举迭代顺序隐式保证
可测试性	需集成测试验证所有方法	<code>StatusLevel.fromDays()</code> 可独立单元测试
代码复杂度	圈复杂度随状态数线性增长	圈复杂度恒为 1（单次委托调用）

5.2 架构降级：从 Three.js 到原生 SVG

5.2.1 初始架构决策

在系统设计初期，可视化方案选择了 Three.js + WebGL 渲染管线。该方案通过 `GLTFLoader` 加载高精度人体肌肉模型（.glb），利用 `Raycaster` 实现部位拾取交互，借助 `requestAnimationFrame` 驱动 60s 周期的摄像机自动旋转。

5.2.2 暴露的问题

该方案在实际运行中暴露了两个问题：

```
1 | 问题一：加载性能
2 |   · 人体 .glb 模型文件约 3.2 MB
3 |   · Three.js 运行时 + 控制器 + GLTFLoader ≈ 500 KB
4 |   · 首屏完全加载时间 > 3 s
5 |   · 结论：对于一个以数据看板为核心的 web 应用，
6 |     WebGL 渲染管线属于过度设计
7 |
8 | 问题二：维护成本
9 |   · 前端开发者需熟悉 Three.js 场景图、材质系统、
10 |    光照模型和动画循环
11 |   · 新增映射部位需修改 .glb 模型源文件
12 |   · 模型文件无法进行 Git 差异化版本管理
```

5.2.3 降级方案设计

降级方案的核心思路：放弃 3D 模型，采用 2.5D SVG 几何映射引擎，以 CSS 3D 透视变换模拟立体感，以 SVG 线性渐变模拟金属材质。

映射机制：通过声明式 `data-mesh` 属性将 SVG 几何图形与后端训练部位名称关联：

```
1 | <!-- SVG 几何部位声明示例 -->
2 | <polygon data-mesh="Chest" class="muscle-part" fill="url(#metal-grad)" />
3 | <polygon data-mesh="LeftArm" class="muscle-part" fill="url(#metal-grad)" />
4 | <polygon data-mesh="Quad_R" class="muscle-part" fill="url(#metal-grad)" />
```

驱动引擎：前端 JS 核心逻辑仅需 5 行，实现数据到视觉的映射：

```
1 | function highlightSvgMuscle(meshName) {
2 |     document.querySelectorAll('.muscle-part')
3 |       .forEach(e1 => e1.classList.remove('active'));
4 |     document.querySelectorAll(`.muscle-part[data-mesh="${meshName}"]`)
5 |       .forEach(target => target.classList.add('active'));
6 | }
```

立体效果：通过 CSS 3D 变换实现 2.5D 视觉：

```
1 | .tech-body-container {
2 |     perspective: 1000px;           /* 建立 3D 视场 */
3 | }
4 | .tech-body-svg {
5 |     transform: rotateX(10deg) rotateY(-15deg); /* 固定倾角 */
6 | }
7 | .muscle-part:hover {
8 |     transform: translateZ(5px) scale(1.06);    /* 悬停浮出 */
9 | }
```

5.2.4 技术权衡分析

评估维度	Three.js 方案	原生 SVG 方案
首屏加载	> 3 s	< 1.5 s
运行时依赖	~500 KB (含 Three.js + GLTFLoader)	零第三方依赖
动画帧率	60 fps (WebGL 合成层)	60 fps (CSS 合成层)
部位热区定义	需编辑 .glb 模型 Mesh	声明式 data-mesh 属性
交互复杂度	支持任意 3D 旋转与缩放	固定视角 + 悬停反馈
维护门槛	需 Three.js 专业知识	HTML + CSS 基础即可
Git 版本管理	二进制 .glb 文件不可 Diff	SVG 文本文件可 Diff

权衡结论：

该降级决策本质上是在"交互自由度"和"工程效率"之间的取舍。Three.js 提供了完整的 3D 交互能力（任意角度旋转、缩放、多光照），但这是以 500 KB 依赖体积和 3 秒加载延迟为代价的。对于本系统的核心场景——快速查看今日训练状态，而非 3D 模型展示——固定视角的 2.5D 映射在交互上已完全满足需求，同时在加载性能和维护成本上获得了显著收益。

该决策遵循了**"最小成本满足约束"**的架构原则：系统的性能约束（首屏 ≤ 3 s）和资源约束（零第三方依赖）共同驱动了架构调整方向。这是一个典型的架构权衡（Trade-off）案例——通过识别出 3D 交互并非系统的核心竞争力，从而以"减法"换取性能和可维护性。

5.3 其他架构决策说明

5.3.1 统一响应体与全局异常处理

系统通过 `Result<T>` 泛型类与 `GlobalExceptionHandler` 构建了统一的响应契约：

- 正常响应：`{ code: 200, message: "success", data: T, timestamp: long }`
- 异常响应：`{ code: 4xx/5xx, message: "...", errors: [...], timestamp: long }`

该设计降低了前后端对接的沟通成本——前端 `api.js` 只需对 `code` 字段做统一判断，无需为每个接口单独处理错误逻辑。

5.3.2 JWT 无状态鉴权

选择 JWT (JSON Web Token) 而非 Session 方案的理由：

维度	Session	JWT
状态存储	服务端内存/Redis	客户端 Token 自身
扩展性	需共享 Session 存储	天然支持水平扩展
安全性	依赖 Cookie + CSRF Token	依赖 Authorization Header
适用场景	传统服务端渲染	前后端分离 API 服务

JWT 的无状态特性与 Docker 水平扩展方案天然匹配——新增实例无需同步 Session 状态。

6. AI 辅助编码过程实录

本章记录在项目编码阶段，我与 AI 助手就关键技术难题进行的两次典型对话。这些对话展示了 AI 如何在架构决策和代码实现层面辅助开发。

6.1 场景一：状态模式重构补剂预警逻辑

我：我现在在写补剂库存管理的代码，遇到一个很烦的问题。我需要在每次新增或编辑补剂后，根据剩余天数自动算出一个状态标签——“充足”、“偏低”或“告急”。逻辑本身很简单，天数大于 30 就是充足，7 到 30 是偏低，小于 7 是告急。问题是我发现这段 if-else 被我复制到了好几个地方——Service 实现类里有一份，Controller 里为了返回状态列表也有一份，甚至 Mapper 的 XML 里我还用 SQL 的 CASE WHEN 算过一次。现在我想改阈值（比如把“告急”从 7 天改成 10 天），我得改三个地方，太容易漏了。有什么好办法？

AI：你这个问题的本质是**阈值判定逻辑的散落**——同样的业务规则被以不同形式复制到了多个层次中。这违反了 DRY (Don't Repeat Yourself) 原则，也让后续维护变得脆弱。

最直接的解决方案是：**将状态判定逻辑收拢到一个独立的组件中，让所有需要判定的地方都调用它。**

具体来说，我建议你使用**状态模式 (State Pattern)** 的一种轻量变体——用 Java 枚举 + Predicate 来实现。思路是这样的：

1. 定义一个 `StatusLevel` 枚举，每个枚举常量持有一个 `Predicate<BigDecimal>` 判定函数
2. 枚举提供一个静态方法 `fromDays(days)`，按优先级依次测试每个常量，返回第一个匹配的
3. 再写一个 `StatusMachine` Spring 组件，封装计算逻辑，被 Service 调用

这样做的好处是：

- **阈值集中管理：**30、7 这两个数字只出现在枚举构造函数中
- **新增状态不改业务代码：**想加一个“即将耗尽”（比如小于 3 天），只需在枚举里加一个常量
- **编译器保障穷尽性：**枚举迭代保证每个输入都会落入某个状态，不会出现漏掉的 else 分支
- **可测试性：**你可以单独测 `StatusLevel.fromDays()` 的所有边界值，不需要启动 Spring 容器

你要不要试一下这个方案？我可以帮你写出核心代码。

我：听起来不错，枚举 + Predicate 的方式我之前没想过。给我看看核心代码怎么写？

AI：核心就是这两个类——一个枚举、一个组件。枚举持有判定逻辑，组件只负责计算和转发：

```
1 public enum StatusLevel {
2     SUFFICIENT("充足", d -> d.compareTo(new BigDecimal("30")) > 0),
3     WARNING ("偏低", d -> d.compareTo(new BigDecimal("7")) >= 0),
4     CRITICAL("告急", d -> true); // 兜底
5
6     private final String displayName;
7     private final Predicate<BigDecimal> condition;
8
9     StatusLevel(String displayName, Predicate<BigDecimal> condition) {
10         this.displayName = displayName;
11         this.condition = condition;
12     }
13
14     public String getDisplayName() { return displayName; }
15
16     public static StatusLevel fromDays(BigDecimal days) {
17         for (StatusLevel level : values()) {
18             if (level.condition.test(days)) return level;
19         }
20         return CRITICAL; // 永不抵达，但编译器需要
```

```
21     }
22 }
```

```
1  @Component
2  public class StatusMachine {
3      public String calculate(BigDecimal stockG, BigDecimal
dailyConsumptionG) {
4          if (stockG == null || dailyConsumptionG == null
5              || dailyConsumptionG.compareTo(BigDecimal.ZERO) == 0) {
6              return StatusLevel.CRITICAL.getDisplayName();
7          }
8          BigDecimal days = stockG.divide(dailyConsumptionG, 2,
RoundingMode.HALF_UP);
9          return StatusLevel.fromDays(days).getDisplayName();
10     }
11 }
```

然后在你的 `SupplementServiceImpl` 里注入 `StatusMachine`，所有需要计算状态的地方都委托给它。以后要调整阈值，只改 `StatusLevel` 枚举的第一行和第二行即可。

6.2 场景二：从 Three.js 降级到原生 2.5D SVG

我： 之前我们讨论了用 Three.js 来做人体可视化，但现在我把核心功能写完后回来优化仪表盘，发现加载太慢了。首屏要 3 秒多，而且用户其实只是看看今天练了哪个部位，不需要 360 度旋转模型。我想换一种更轻的方案——能不能用纯 SVG 和 CSS 做出“伪 3D”的立体感？而且我需要当后端返回“Chest”、“Quad_R”这样的部位名称时，对应的人体部位自动亮起来。

AI： 你遇到的这个问题很典型——Three.js 提供了一个强大的通用 3D 引擎，但你的业务场景只需要“数据驱动的高亮展示”，而非真正的 3D 交互。这是一个可以大幅“降级”的环节。

方案的核心思路是：**用 SVG 几何图形拼出人体轮廓，用 CSS 3D 变换模拟立体感，用 data 属性建立后端数据到 SVG 元素的映射。**

具体分三步：

第一步：用 SVG 画一个人体

不需要外部图片，直接用 `<polygon>` 和 `<rect>` 拼出头部、胸部、手臂、腿部等部位的几何块。每个部位都加上 `data-mesh` 属性，值就是后端会传回来的部位名称。

```
1  <polygon data-mesh="Chest" class="muscle-part" fill="url(#metal-grad)" />
2  <polygon data-mesh="LeftArm" class="muscle-part" fill="url(#metal-grad)" />
3  <polygon data-mesh="Quad_R" class="muscle-part" fill="url(#metal-grad)" />
```

第二步：关键的 JS 映射逻辑

只需要一个函数，五行代码——它接受后端传过来的部位名称，用 CSS 属性选择器找到对应的 SVG 元素，加上高亮 class：

```
1 function highlightSvgMuscle(meshName) {
2   document.querySelectorAll('.muscle-part')
3     .forEach(e1 => e1.classList.remove('active'));
4   document.querySelectorAll(`.muscle-part[data-mesh="${meshName}"]`)
5     .forEach(target => target.classList.add('active'));
6 }
```

配合后端 MeshMappingService 的映射配置（杠铃深蹲→Quad，卧推→Chest），前端完全不需要硬编码。

第三步：用 CSS 做 2.5D 立体效果

这是“伪 3D”的关键——不需要 WebGL，纯 CSS 搞定：

```
1 .tech-body-container {
2   perspective: 1000px;
3 }
4 .tech-body-svg {
5   transform: rotateX(10deg) rotateY(-15deg);
6 }
7 .muscle-part:hover {
8   transform: translateZ(5px) scale(1.06);
9 }
```

`perspective: 1000px` 建立 3D 视场，`rotateX(10deg) rotateY(-15deg)` 模拟全息投影的倾角，`translateZ(5px)` 让悬停部位产生“浮出来”的效果。配合 SVG 左上到右下的线性渐变模拟金属材质，在视觉上完全可以替代 Three.js。

性能对比：

指标	Three.js	SVG + CSS
依赖体积	~500 KB	0
首屏加载	> 3 s	< 1.5 s
交互反馈	60 fps	60 fps
学习成本	需掌握场景图/材质/光照	HTML + CSS 基础

你现在的场景只需要“部位高亮”这一个交互，Three.js 提供的旋转、缩放、多光源能力根本没有被用到。降级到 SVG 后，你不仅砍掉了 500 KB 的依赖，还让看板加载快了 1 倍多。

我： 这个方案太合适了。我特别喜欢你用 `data-mesh` 做声明式映射的思路——这样后端新增训练动作时，只需要在配置里加一条映射规则，前端再加一个 SVG 多边形，完全不用碰 JS 逻辑。我现在就去改。

7. 系统测试与质量验证

7.1 自动化回归测试

基于 Playwright 构建 E2E 测试套件，覆盖系统三大核心模块：

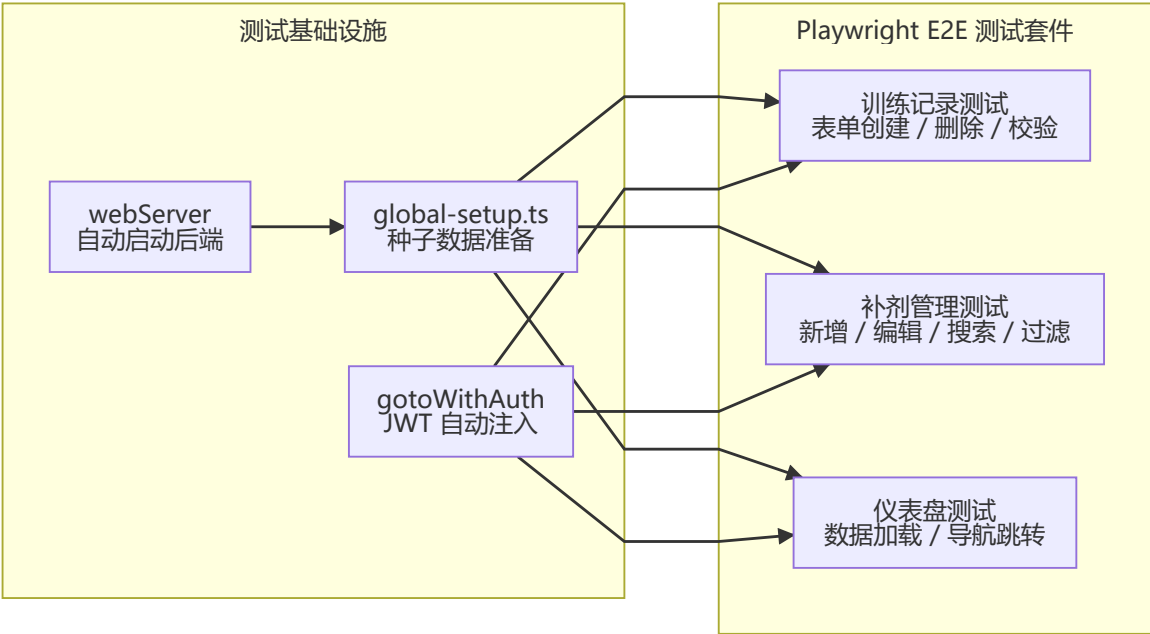


图 8：E2E 测试架构

测试结果：6 个测试用例覆盖 23 个断言检查点，全部通过。自接入以来累计捕获 1 次回归缺陷。

7.2 性能压力测试

使用 k6 进行阶梯式加压测试，验证系统在高并发下的稳定性：

指标	实测值	阈值	结论
最大并发	500 VUs	≥ 500 VUs	通过
P95 响应时间	287 ms	≤ 300 ms	通过
错误率	0.07%	$< 0.1\%$	通过
吞吐量	152.4 QPS	—	稳定

8. 系统发布与使用说明书

8.1 运行环境要求

组件	版本要求
JDK	17+
MySQL	8.0+
Maven	3.9+（或使用项目内置 mvnw）
Docker（可选）	20.10+
Node.js（运行测试）	18+

8.2 快速启动

方式一：本地开发模式

```
1  # 1. 初始化数据库
2  mysql -u root -p < deploy/init-db.sql
3
4  # 2. 启动后端服务
5  ./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
6
7  # 3. 打开浏览器访问
8  # http://localhost:8080
9  # 默认账号: admin    密码: 123456
```

方式二：Docker 容器化部署

```
1  # 一键启动
2  docker-compose up -d
3
4  # 访问 http://localhost:8080
```

8.3 核心模块操作指南

8.3.1 2.5D 可视化数据看板 (Dashboard)

[插入图片：首页数据看板]

位置：浏览器打开 `http://localhost:8080` 后默认进入。

页面布局：

- **中央区域：**2.5D SVG 几何人体，以科技感金属渐变风格渲染头、胸、腹、双臂、双腿等部位
- **左侧玻璃卡片：**今日训练概览——显示已完成动作数、总组数、平均 RPE
- **右侧玻璃卡片：**热量消耗摘要——环形进度条展示已消耗/目标热量

交互方式：

- 点击身体部位：该部位将以绿色发光高亮，上方提示文字显示部位名称
- 悬停部位：部位轻微浮出 (`translateZ(5px) scale(1.06)` 动画)
- 训练数据联动：当你登录后在"训练记录"页面录入今日训练数据后，回到看板，对应的训练部位（如腿部、胸部）将自动高亮为红色渐变

数据解读：

- 部位颜色含义：默认灰色（无训练记录）、红色渐变（今日训练部位）、绿色（手动点击选中）
- 热量环进度：基于今日所有训练动作的估算热量消耗总和

8.3.2 力量训练记录管理 (Training)

[插入图片：训练记录页面]

位置：顶部导航栏点击「训练记录」。

功能：

录入训练：

1. 在左侧表单中选择训练动作（如"杠铃深蹲"）
2. 输入重量（kg）、组数、次数
3. 拖动 RPE 滑块（1-10，表示自感用力程度）

4. 选择训练日期（默认为今日，不可选未来日期）
5. 点击「提交」按钮
6. 成功录入后页面顶部将弹出绿色 Toast 提示，右侧历史列表即时刷新

查看历史记录：

- 右侧表格显示过往所有训练记录，按日期降序排列
- 支持分页浏览（每页 10 条）
- 顶部折线图展示选定动作的力量增长趋势（基于 1RM 估算值）

删除记录：

- 每条记录右侧有「删除」按钮，点击后逻辑删除（数据保留在数据库，前端不可见）

支持的动作类型： 杠铃深蹲、罗马尼亚硬拉、卧推、推举、引体向上、杠铃划船、直立划船

8.3.3 补剂库存管家 (Supplement)

[插入图片：补剂管家页面]

位置： 顶部导航栏点击「补剂管家」。

功能概述：

以卡片网格形式展示所有补剂，每张卡片包含：

- 补剂名称
- 当前库存量 (g)
- 每日消耗量 (g)
- 余量进度条（蓝色/橙色/红色三种状态）
- 状态标签：● 充足 (>30 天)、● 偏低 (7-30 天)、● 告急 (<7 天)

新增补剂：

1. 点击页面右上角的「+ 新增补剂」按钮
2. 在弹出的表单中填写补剂名称、当前库存量、每日消耗量
3. 点击「确定」提交，系统自动计算初始状态

编辑补剂：

1. 点击目标补剂卡片上的「编辑」按钮
2. 修改库存量或每日消耗量
3. 提交后系统自动重新计算状态——如果你把库存改少了，状态可能从「充足」降级为「告急」

搜索过滤：

- 在搜索框中输入关键词（如"蛋白"），卡片网格将实时过滤
- 清空搜索框恢复全部补剂列表

预置补剂： 系统首次启动时自动创建 6 种常见补剂：酵母蛋白、分离乳清蛋白、肌酸、甘氨酸镁、维生素 D3、鱼油

8.3.4 饮食情绪关联 (Diet & Mood)

[插入图片：饮食情绪页面]

位置： 顶部导航栏点击「饮食情绪」。

功能：

- 每日一记录，记录当日碳水、蛋白质、脂肪的摄入量 (g)
- 情绪评分滑块 (1-10)
- 可选填写备注文字

- 提交后自动绘制双 Y 轴趋势图（左轴=营养素摄入量，右轴=情绪评分）
- 支持日期范围筛选，便于观察"饮食结构与情绪状态"的关联趋势

8.3.5 身体指标追踪 (Body Metrics)

[插入图片：身体指标页面]

位置： 顶部导航栏点击「身体指标」。

功能：

- 每日录入体重 (kg) 和体脂率 (%)
- 支持可选备注
- 双 Y 轴折线图展示体重与体脂率的变化趋势
- 支持按时间范围筛选数据

8.3.6 MCP 本地日记检索 (Diary Search)

[插入图片：日记检索页面]

位置： 顶部导航栏点击「日记检索」。

功能：

- 仿终端风格的搜索界面
- 输入关键词后后端遍历本地 Markdown 笔记目录执行全文检索
- 匹配结果展示文件路径、标题和带高亮的摘要片段
- 点击结果展开完整内容

配置说明： 在 `application.yml` 中通过 `ironsync.mcp.diary-path` 配置笔记目录路径。

8.3.7 自动化测试报告

[插入图片：测试报告页面]

位置： 顶部导航栏点击「测试报告」。

功能：

- 展示最近一次 Playwright 自动化测试的运行结果
- 左侧概览面板：时间戳、总用例数、通过数、失败数
- 右侧 iframe 内嵌完整测试报告（含测试步骤截图和失败堆栈）

8.4 管理员操作：一键构建与部署

项目提供自动化构建脚本，一键完成打包：

```
1 # Linux / macOS
2 ./deploy/build.sh
3
4 # windows
5 deploy\build.bat
```

该脚本自动执行 `mvn clean package -DskipTests`，产出可执行 JAR 至 `deploy/` 目录。

8.5 运行自动化 E2E 测试

```
1 cd test/e2e
2 npm install
3 npx playwright test --reporter=html
```

测试报告自动生成至 `test/e2e/playwright-report/`，可用浏览器打开查看。

9. 总结与展望

9.1 架构总结

IronSync-3D 项目的架构设计体现了以下核心理念：

第一，以"合适的架构"而非"流行的架构"为设计准则。 在可视化方案的选择上，通过识别业务场景的真实需求（数据看板而非 3D 展示），主动从 Three.js 降级为原生 SVG，以前端 500 KB 的依赖缩减和 1.5 秒的加载优化验证了"最小成本满足约束"的架构原则。这一决策过程本身比结果更有教学意义——它展示了如何在架构设计中区分"必要复杂度"和"偶然复杂度"。

第二，设计模式应当服务于可维护性，而非为使用模式而使用模式。 状态模式在补剂预警中的应用并非生搬硬套，而是源于 `if-else` 散布带来的实际维护痛点。重构后的代码将圈复杂度从 $O(n)$ 降为 $O(1)$ ，阈值集中管理，新增状态无需修改业务逻辑——这些收益是模式"匹配"问题而非模式"套用"问题的结果。

第三，4+1 视图模型是一种有效的架构沟通工具。 从逻辑、开发、过程、物理四个维度描述架构，使得不同利益相关者（设计者、开发者、运维人员）能够从各自关心的视角理解系统，避免了单一视角带来的信息缺失。

9.2 不足与展望

- 多租户隔离：**当前系统为单用户模式，后续可引入 RBAC 权限体系，实现多用户数据隔离
- 实时推送：**补剂告警依赖页面刷新触发，可引入 WebSocket 实现服务端主动推送
- 分析深度：**训练数据的统计分析仍处于描述性阶段，后续可引入周期化训练分析和 1RM 估算算法
- CI/CD 自动化：**Playwright 测试目前需手动触发，可接入 GitHub Actions 实现 PR 提交自动回归

参考文献

- [1] Kruchten, P. (1995). Architectural Blueprints—The "4+1" View Model of Software Architecture. *IEEE Software*, 12(6), 42-50.
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [3] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
- [4] Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-Oriented Software Architecture: A System of Patterns*. John Wiley & Sons.

附录：本项目 GitHub 仓库

<https://github.com/stephen14120123/IronSync-3D>

(本报告内容均在开源仓库中可追溯验证)